



A Unified Post-Quantum Transport for Lux: QUIC-over-ZAP, One Port per Validator

Zach Kelling

Lux Industries

2026-06-30 (LP-202, Draft — Design & Specification)

Abstract

We specify a single post-quantum transport that collapses a Lux validator’s three network endpoints — 9630 HTTP-RPC, 9631 classical staking-P2P, and 9632 ZAP-native RPC — onto *one* UDP port carrying QUIC over ZAP. Three stream classes (RPC, consensus-P2P, ZAP-native services) are multiplexed on one connection, eliminating cross-class head-of-line blocking. Every connection is end-to-end post-quantum: confidentiality and key agreement come from the IANA hybrid group X25519MLKEM768 (0x11EC) in the QUIC TLS 1.3 handshake, and peer authentication is a hybrid *classical* $\wedge$ *ML-DSA-65* validator identity that preserves the existing 20-byte hash-of-hash `NodeID` while binding a FIPS-204 signing key channel-bound to the QUIC session.

This is a *transport* change only. The Lux consensus message set (the P2P message set Lux inherited from its Avalanche fork) and the BFT decision logic are unchanged byte-for-byte; consensus messages ride QUIC streams instead of a shared TCP byte stream. The proven classical TCP staking path (9631) remains the negotiated fallback: validators upgrade *to* QUIC-PQ and never retire the classical path until a devnet A/B test proves *byte-identical finality*. We give the wire format, the hybrid handshake and its channel binding, the message-to-stream map, capability negotiation for mixed fleets, the per-network DNS scheme (`<NN>.lux.network`), the A/B acceptance methodology, and an argument that the transport change does not alter $f < n/3$ safety or liveness. The design reuses `luxfi/zap`’s QUIC + X25519MLKEM768 transport, its SPEC-ZAP-PQ-v1 (“Z-Wing”) hybrid handshake, `luxd`’s already-present ML-DSA-65 `NodeID` scheme, and the `node2` leaderless BLS vote-mesh — nothing is forked. We are explicit throughout about what is *proven and live* versus what is *specified here and not yet built*.

Contents

1	Introduction	4
1.1	Design tenets	4
1.2	Contributions	4
2	Threat model and goals	5
2.1	Adversary	5
2.2	Security goals	5
3	What is already proven and live	5
3.1	QUIC + X25519MLKEM768 transport	6
3.2	The SPEC-ZAP-PQ-v1 (“Z-Wing”) handshake	6
3.3	<code>luxd</code> ’s ML-DSA-65 <code>NodeID</code> scheme and <code>PQHandshake</code>	6
3.4	The <code>node2</code> leaderless BLS vote-mesh	7
3.5	<code>luxd</code> ’s ZAP-native RPC	7
4	The composition gap: what this document adds	7
4.1	Two ZAP lineages	7
5	Wire: one port, three stream classes	8
5.1	Endpoint collapse	8
5.2	Stream classes and demultiplexing	8
5.3	No head-of-line blocking	9
5.4	0-RTT, migration, and their hazards	9
5.5	UDP at the edge	9
6	Post-quantum handshake and hybrid identity	10
6.1	Two layers, bound together	10
6.2	Channel binding	10

6.3	The hybrid <code>NodeID</code>	10
6.4	Handshake flow (design)	11
7	Consensus-P2P over QUIC-ZAP	11
7.1	The message set is unchanged	11
7.2	Drop-in compatibility via the <code>net.Conn</code> seam	12
7.3	Reusing the <code>node2</code> vote-mesh	12
8	Fallback negotiation and mixed fleets	12
8.1	Baseline and capability	12
8.2	Upgrade rule	12
8.3	Migration discipline	13
9	DNS scheme and validator onboarding	13
9.1	One name per validator per network	13
9.2	Local-fiber / NAT validator join	13
10	Devnet A/B acceptance: byte-identical finality	14
11	BFT preservation argument	14
12	Minimal proof-of-concept	15
13	Related Lux work	15
14	Conclusion	15

1 Introduction

A Lux validator today speaks three network dialects on three sockets:

- `:9630` — HTTP/JSON-RPC for the `/ext/*` client API (`DefaultHTTPPort`).
- `:9631` — a single long-lived TCP stream per peer carrying *all* Lux P2P and consensus traffic (`DefaultStakingPort`), authenticated by a TLS 1.3 staking certificate.
- `:9632` — an opt-in ZAP-native RPC listener (`ZAP_RPC_LISTEN`) that re-serves the same `/ext/*` handler chain over the ZAP wire codec.

Three ports means three firewall holes, three TLS stances, three DoS surfaces, three sets of health checks, and — most importantly — *no shared post-quantum posture*. The staking TCP stream also suffers transport head-of-line (HOL) blocking: a slow bootstrap `GetAncestors` transfer and a latency-critical `PushQuery` share one ordered byte stream, so a stalled fetch delays a consensus vote.

This document specifies the endpoint of a different design: **one UDP port per validator**, QUIC over ZAP, end-to-end post-quantum, with RPC, consensus-P2P, and ZAP-native services multiplexed as independent QUIC streams. The change is deliberately confined to the transport. We do not touch the consensus engine, the proposer VM, the message definitions, or the BFT decision rule. Consensus messages are the same bytes; they simply travel on a QUIC stream rather than a TCP segment.

1.1 Design tenets

1. **Decompect transport from consensus.** The BFT logic already sits behind a transport-agnostic seam (the `Upgrader` and `EndpointDialer` interfaces, §7.2). A new transport is a new implementation of that seam, not a change to the seam’s consumers.
2. **Extend, do not break.** TCP staking on 9631 stays as the proven baseline. QUIC-PQ is a *capability* a validator negotiates *up* to. Mixed fleets interoperate; there is no flag day.
3. **Reuse, do not fork.** Every cryptographic and wire primitive already exists in tree: `luxfi/zap`’s QUIC transport with `X25519MLKEM768` (§3.1); its `SPEC-ZAP-PQ-v1` hybrid handshake (§3.2); `luxd`’s ML-DSA-65 `NodeID` scheme and `PQHandshake` (§3.3); and `node2`’s leaderless BLS vote-mesh (§3.4). This paper is mostly *composition*.
4. **Honesty about maturity.** §3 enumerates what is proven and live. Everything else is labelled *[design]*. The acceptance gate (§10) is the line a build must cross before the fallback is retired.

1.2 Contributions

- A one-port wire model with a three-class stream demultiplexer and an analysis of the 0-RTT, connection-migration, and no-HOL-blocking properties (§5).
- A hybrid post-quantum handshake: `X25519MLKEM768` for the confidential channel plus a channel-bound ML-DSA-65 identity proof that preserves the 20-byte classical `NodeID` and upgrades it to a *classical* $\wedge$ *PQ* value (§6). This closes a real gap: the shipped QUIC transport authenticates the *channel* (TLS cert) but exchanges the ZAP `NodeID` as an *unauthenticated* string (§4).
- A byte-preserving map of the full Lux/Simplex message set onto QUIC stream classes, reusing the `node2` vote-mesh semantics (§7).
- A capability-negotiation scheme for mixed TCP/QUIC fleets (§8), a per-network DNS scheme and a NAT/local-fiber validator join procedure (§9).
- A devnet A/B methodology that gates retirement of the fallback on byte-identical finality (§10), and a proof sketch that the transport swap preserves $f < n/3$ safety and liveness (§11).

2 Threat model and goals

2.1 Adversary

We assume a network adversary with the standard partial-synchrony BFT powers, augmented with a store-now-decrypt-later quantum capability:

- **Network (Dolev–Yao).** Full control of the UDP/IP substrate: observe, drop, reorder, inject, replay, and delay packets; spoof source addresses; mount reflection/amplification and address-migration attacks against QUIC.
- **Byzantine participants.** Up to $f < n/3$ of validator stake is adversarial and may equivocate, withhold, or send arbitrary (but well-typed) messages. This is the *message layer’s* model and is unchanged by this document.
- **Harvest-then-decrypt quantum adversary.** Records all ciphertext now and runs a cryptographically-relevant quantum computer *later*. Confidentiality of *today’s* traffic must survive *tomorrow’s* quantum computer; hence hybrid PQ key agreement is mandatory, not optional.
- **Future quantum forger.** May eventually break ECDLP (Shor), so long-lived *authentication* must not rest on a classical assumption alone.

2.2 Security goals

- G1 **PQ confidentiality with forward secrecy.** Session keys are IND-CCA secure if *either* X25519 *or* ML-KEM-768 is unbroken; compromise of a validator’s long-term signing key does not retroactively decrypt recorded sessions.
- G2 **Hybrid peer authentication.** Impersonating a validator requires forging *both* the classical staking credential *and* an ML-DSA-65 signature. A quantum forger that breaks ECDLP alone cannot impersonate; a lattice break that does not also break the classical leg cannot either. (The “X-Wing pattern” applied to identity.)
- G3 **NodeID continuity.** The value that keys the validator set — the 20-byte hash-of-hash NodeID — is preserved across the transport change so no re-registration or stake-map migration is forced.
- G4 **Channel binding.** An authentication proof captured on one connection cannot be replayed onto another connection, chain, or epoch (defeats relay/MITM and cross-chain replay).
- G5 **Transport-transparent BFT.** The consensus safety and liveness properties proven at the message layer hold unchanged under the new transport (§11).
- G6 **Graceful coexistence.** A QUIC-PQ validator and a TCP-only validator reach the *same finality*; the fleet is never partitioned by capability (§8).

Non-goals. We do not redesign the BFT protocol, do not alter the message schema, do not claim a NIST-certified *threshold* scheme (the identity keys here are single-signer ML-DSA-65 per validator), and do not address application-level censorship resistance beyond what the consensus layer already provides.

3 What is already proven and live

This section is the honest foundation. Every primitive the design composes exists in tree and is exercised by tests; the novelty is the *composition*, catalogued in §4.

3.1 QUIC + X25519MLKEM768 transport

luxfi/zap/quic is a working QUIC transport over quic-go v0.59.1. Its TLS 1.3 configuration (quic/tls.go) sets

$$\text{CurvePreferences} = [\text{X25519MLKEM768 (0x11EC)}, \text{X25519}],$$

so a PQ-capable peer always negotiates the hybrid group first and a legacy peer falls back to X25519. ALPN is pinned to "zap/1" and re-checked post-handshake. The transport already provides:

- one UDP socket per node, up to 1024 bidirectional and 1024 unidirectional concurrent streams (quic/server.go);
- *per-Call streams*: each RPC gets a fresh bidirectional stream carrying exactly one request frame and one response frame, so no correlation header is needed and concurrent calls never serialize (node_quic.go);
- 0-RTT resumption via TLS session tickets, gated by a RejectEarlyData switch for non-idempotent handlers (quic/config.go);
- connection migration and a coordinated DRAIN close;
- a test that asserts the negotiated ConnectionState().CurveID == 0x11EC — i.e. the hybrid KEM is really on the wire.

The frame format on a QUIC stream is [4-byte LE length][ZAP message bytes], with MaxFrameSize = 10MiB (quic/conn.go). This is the confidential, PQ-key-agreed substrate. What it does *not* yet do is bind an ML-DSA identity to the channel — see §4.

3.2 The SPEC-ZAP-PQ-v1 (“Z-Wing”) handshake

luxfi/zap/handshake implements a transport-agnostic hybrid handshake over any io.ReadWriter (LP-9702, “Z-Wing”). Suite 0x01 = X25519MLKEM combines an X25519 exchange and an ML-KEM-768 encapsulation; the two shared secrets are fed to a labelled, length-tagged concatenation combiner keyed by the running transcript hash H_2 (SHA3-256):

$$\text{PRK} = \text{HKDF-Extract}(\text{salt} = H_2, \text{lbl}_{\text{X25519}} \parallel \text{ss}_{\text{ec}} \parallel \text{lbl}_{\text{MLKEM}} \parallel \text{ss}_{\text{kem}}),$$

from which five session secrets are HKDF-Expanded (directional keys, nonce salts, resumption PSK). Static identities are ML-DSA-65 keypairs; the AUTH step signs $\text{lbl}_{\text{proto}} \parallel \text{suite} \parallel H_2 \parallel \text{lbl}_{\text{role}}$ under FIPS-204 context string "lux-zap-pq-v1". The post-handshake Session is an AES-256-GCM stream with strict monotone nonce counters, epoch rekeying, and ALERT framing. Identity value:

$$\text{PeerID} = \text{SHA3-256}(\text{ML-DSA-65 public key}) \in \{0, 1\}^{256}.$$

This is the exact ML-DSA authentication primitive the QUIC transport is missing; §6 channel-binds it to the QUIC session rather than re-running its KEM (the KEM is already done by TLS).

3.3 luxd’s ML-DSA-65 NodeID scheme and PQHandshake

luxd already generalises NodeID beyond the classical certificate hash. Classically,

$$\text{NodeIDFromCert}(c) = \text{RIPEMD160}(\text{SHA256}(c.\text{Raw})) \in \{0, 1\}^{160},$$

the 20-byte, BTC-style “hash-of-hash” (ids/node_id.go). Under a PQ profile, when a validator has a StakingMLDSAPub, luxd instead derives

$$\text{NodeID} = \text{NodeIDSchemeMLDSA65.DeriveMLDSA}(\text{chainID}, \text{StakingMLDSAPub}),$$

and the TLS certificate becomes *transport-only* (`config/node/config.go`, `staking/pq.go`). A network-layer `PQHandshake` (`network/peer/handshake.go`) already exists: an INIT/RESP exchange using ML-KEM-768/1024 for the session key and ML-DSA-65 to bind the node identity, deriving an AEAD key via cSHAKE256 with customization "NODE_AEAD_V1", chain-ID-bound to prevent cross-chain replay, with `ProfileStrictPQ` / `ProfilePermissive` / `ProfileFIPS` stances. **The hybrid PQ NodeID is therefore not new work** — this document *routes* it over QUIC.

3.4 The node2 leaderless BLS vote-mesh

`node2` (tag `v0.1.1`) hosts the `consensus2` engine and disseminates its votes over a full mesh of real sockets framed by the canonical ZAP wire codec. Five validator *processes* on loopback independently reach a $> 2/3$ -stake BLS quorum-certificate finality over the wire (`node2_cluster_test`, `cluster_5.sh`). The proven wire semantics we reuse:

- **Frame:** [4-byte BE length] [1-byte msg_type] [payload], `MaxMessageSize` = 16 MiB (`lux/zap/wire.hpp`). The low 6 bits of `msg_type` are the service id (`0x00..0x3F`); bit 7 (`0x80`) is a response flag, bit 6 (`0x40`) an error flag.
- **Vote codec:** a `SignedVote` encodes as `block_id(32) || voter_pubkey(48) || sig(96)` under `kVoteMsgType` = `0x11`; `decode_vote` rejects any wrong-width field *and any trailing bytes*, so a peer cannot inject a malformed or malleable vote (`consensus2/zap/vote_codec.hpp`).
- **Non-blocking reassembly:** `FrameReader` accumulates `recv(MSG_DONTWAIT)` bytes and yields a frame only when complete, so one slow peer never stalls a single-threaded pump over N peers (`node2/frame_reader.hpp`). This is exactly the discipline QUIC gives us for free per stream.

3.5 luxd's ZAP-native RPC

`luxd` serves an opt-in ZAP-native RPC listener (`server/http/zap_listener.go`) bound from `ZAP_RPC_LISTEN` (§1, port 9632 by convention). It wraps the *same* `/ext/*` handler chain as the HTTP listener in the `zap-proto/http` wire encoding, request/response Call semantics. Note the two ZAP lineages in play (§4.1): the RPC listener uses `zap-proto/http`; the transport in this document uses `luxfi/zap` QUIC. Unification means moving RPC onto the same `luxfi/zap` per-Call streams.

4 The composition gap: what this document adds

Precisely one security-relevant thing is missing, plus wiring. The shipped QUIC transport's peer-identity step (`performServerHandshake` / `writeHandshake` in `quic/conn.go`) exchanges a fixed 64-byte frame whose payload is the NodeID *as a UTF-8 string*. The source comment is candid: this “mirrors node.go's TCP handshake exactly” and, as in `node2`'s index handshake, the asserted id is “unauthenticated but not security-load-bearing.” For a *consensus* transport that is not acceptable: nothing binds the asserted NodeID to the TLS channel or to any signing key. The channel is PQ-confidential (X25519MLKEM768) and channel-authenticated to the TLS *certificate*, but the ZAP NodeID rides on trust.

4.1 Two ZAP lineages

For the avoidance of confusion the estate contains two “ZAP” stacks. (i) `github.com/luxfi/zap` — the pure transport with the QUIC and Z-Wing PQ handshake used *as the transport* here. (ii) `zap-proto/*` (`zap-proto/http`, `zap-proto/go`) — the Cap'n-Proto-style service-RPC wire used by `luxd`'s `ZAP_RPC_LISTEN` and by the C++ `zap-cpp-core` framing that `node2` uses. They share a frame shape by design but are distinct modules. This document's transport is `luxfi/zap`

Capability	Status	Where
QUIC 1-port, streams, migration, 0-RTT	<i>live</i>	luxfi/zap/quic
X25519MLKEM768 PQ key agreement	<i>live</i>	quic/tls.go, CurveID test
Hybrid ML-DSA-65 handshake (KEM+AUTH)	<i>live</i>	zap/handshake (SPEC-ZAP-PQ-v1)
ML-DSA-65 NodeID scheme, PQHandshake	<i>live</i>	luxd staking/pq.go, network/peer
Leaderless BLS vote-mesh over ZAP frames	<i>live</i>	node2, consensus2
ZAP-native RPC over zap-proto/http	<i>live</i>	luxd server/http
Channel-bound ML-DSA identity <i>on QUIC</i>	<i>[design]</i>	§6
Three-class stream demux on one conn	<i>[design]</i>	§5
QUIC EndpointDialer+Upgrader in luxd	<i>[design]</i>	§7.2
Capability negotiation / mixed fleet	<i>[design]</i>	§8
Per-network DNS + local-fiber join	<i>[design]</i>	§9
A/B byte-identical-finality harness	<i>[design]</i>	§10

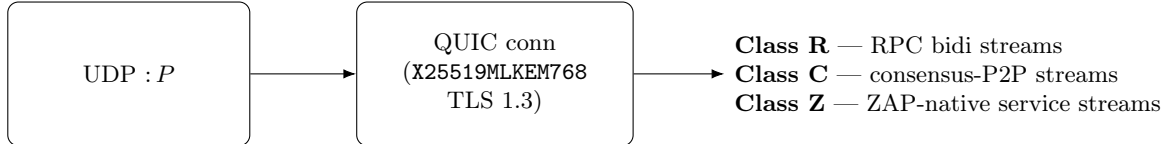
Table 1: Proven-and-live versus specified-here. The design work is composition and wiring, not new cryptography.

QUIC; the RPC *payloads* it carries may be either lineage’s messages, since above the stream both are just length-prefixed frames.

5 Wire: one port, three stream classes

5.1 Endpoint collapse

A validator binds exactly one UDP port P (default 9631, so the QUIC-PQ endpoint reuses the staking port number on UDP while the legacy TCP staking listener keeps the same number on TCP — see §8). All three former endpoints become stream classes on the one QUIC connection to each peer:



5.2 Stream classes and demultiplexing

QUIC gives independent, individually reliable+ordered streams. We assign:

Class R (RPC) The `/ext/*` client API. *One bidirectional stream per Call*: write one request frame, read one response frame, close. Replaces both `:9630 HTTP` and `:9632 ZAP-RPC`. Idempotent read RPCs MAY use 0-RTT; all others set `RejectEarlyData` (§5.4).

Class C (consensus-P2P) The Lux/Simplex message set (§7). Request/response consensus exchanges (`Get/Put`, `GetAncestors/Ancestors`, `PushQuery/PullQuery`→`Chits`) map to bidirectional streams; pure gossip (`Ping`, `Pong`, `PeerList`, `AppGossip`, `Simplex Vote/Notarization`) maps to unidirectional streams. *Never* 0-RTT.

Class Z (ZAP-native) Native ZAP services (`rpcdb`, `rpcvm`, `DA`, `messaging`, and the `node2` vote frames). Framed `[4-BE 1en][1 msg_type][payload]` with `msg_type` demultiplexing to the service handler exactly as in `zap-cpp-core`.

Demux mechanism. We do *not* overload the reserved flag bits of the ZAP `msg_type` byte (`0x80/0x40`) for class selection — those keep their response/error meaning. Instead the stream class is determined structurally and cheaply:

1. **Directionality.** Unidirectional streams are always gossip (Class C uni or Class Z one-way); bidirectional streams are request/response (Class R or Class C req/resp or Class Z Call).
2. **A 1-byte stream-class prefix** is written as the first byte of the first frame on every stream: 0x52 'R', 0x43 'C', 0x5A 'Z'. The reader peeks one byte, routes the stream to the class dispatcher, and the remainder of the stream is that class's native framing. This keeps the three message schemas fully independent (orthogonal): Class C stays byte-identical to the Lux message encoding, Class Z stays byte-identical to **zap-cpp-core**, and neither needs to know the other exists.

The class prefix is the *only* new byte on the wire, and it lives below all three payload schemas, so no existing message encoding changes.

5.3 No head-of-line blocking

On the TCP staking stream, a lost segment stalls *all* subsequent bytes until retransmission, and a large in-flight **Ancestors** transfer delays every message queued behind it. QUIC loss recovery and flow control are *per stream*: a lost packet on the bootstrap stream does not delay a **PushQuery** on another stream, and a 10 MB **Ancestors** response cannot starve a 200 B **Chits**. This strictly improves consensus message latency under load and, as §11 argues, can only help liveness. It is the same property **node2**'s **FrameReader** engineers by hand for its single-threaded pump — here it is a transport guarantee.

5.4 0-RTT, migration, and their hazards

0-RTT (early data). QUIC resumption lets a returning client send application data in the first flight, saving a round trip. Early data is *replayable* by an active adversary ([11] §9.2). Policy:

- Class R: permitted only for provably idempotent read RPCs; the server sets **RejectEarlyData** for every mutating handler.
- Class C: **forbidden**. Consensus messages are never carried as 0-RTT early data. A replayed **PushQuery** or **Vote** must never be actionable off the resumption path; consensus streams open only after the handshake confirms (1-RTT).
- Class Z: per-service, defaulting to forbidden.

This preserves the BFT model's assumption that a message is delivered because a peer *sent* it, not because an adversary *replayed* it (see G4 and §11).

Connection migration. QUIC identifies a connection by Connection ID, not by 4-tuple, so a validator surviving a NAT rebind or an interface change (mobile/local-fiber uplink flap) keeps its consensus connections alive across the address change. Migration is authenticated by QUIC path validation; the peer identity (§6) is bound at handshake and does not re-open on migration. **RemoteAddr** is therefore informational, not authoritative — the ML-DSA-bound **NodeID** is the identity, never the IP.

5.5 UDP at the edge

QUIC is UDP. Two deployment shapes:

- **Direct public IP** (recommended for validators): the UDP port is reachable directly; no L7 proxy sits in the consensus path. This is the natural fit for §9's one-name-per-validator scheme.
- **L4 UDP load balancer** (e.g. DO LB in UDP mode): must forward UDP with source-address preservation or client-IP affinity; an L7/HTTP LB *cannot* carry QUIC consensus

traffic and must not be placed in the path. This is an explicit ops constraint, not a nice-to-have: QUIC path validation and migration require the LB to be transparent at L4.

Because RPC now shares the QUIC port, the public RPC edge also moves to UDP/QUIC (HTTP/3). A classical HTTPS/H2 edge MAY be kept temporarily as a translation layer for non-QUIC clients (§8), but validator-to-validator traffic is pure QUIC-PQ.

6 Post-quantum handshake and hybrid identity

6.1 Two layers, bound together

The handshake has a *key-agreement* layer and an *authentication* layer, and the novelty is the binding between them.

Layer 1 — Confidential channel (live). The QUIC TLS 1.3 handshake negotiates X25519MLKEM768 (0x11EC). The session secret is IND-CCA if either X25519 or ML-KEM-768 holds ([1, 4, 2]); QUIC keys are forward-secret (ephemeral KEM shares). This satisfies G1. We deliberately use the *IANA* hybrid group, not a Lux-private variant, so every TLS 1.3 stack in the estate speaks the same group.¹

Layer 2 — Hybrid identity (design). Over the first bidirectional stream (the *identity stream*, class-tag 'C', stream id 0), each side runs the ZAP-PQ AUTH exchange (§3.2) *channel-bound* to the QUIC session. This replaces the unauthenticated 64-byte NodeID string (§4).

6.2 Channel binding

Let the completed QUIC/TLS session expose an RFC 5705 exporter [10]. Define the channel-binding value

$$\tau = \text{TLS-Exporter}(\text{label} = \text{"lux-zap-pq-v1 channel-binding"}, \text{context} = \text{chainID}, 32).$$

τ is a 32-byte value that is (i) unique to this connection (depends on the hybrid KEM transcript), (ii) unknown to any party that did not complete *this* handshake, and (iii) scoped to `chainID`. The identity-stream AUTH message is the SPEC-ZAP-PQ-v1 signature over the transcript H_2 with τ folded in:

$$\text{sig} = \text{ML-DSA-65.Sign}_{sk}(\text{lbl}_{\text{proto}} \parallel \text{suite} \parallel H_2 \parallel \tau \parallel \text{lbl}_{\text{role}}), \quad \text{context} = \text{"lux-zap-pq-v1"}.$$

Because τ binds the signature to *this* QUIC session and to `chainID`, an AUTH captured on one connection is worthless on another (G4): a MITM who terminates TLS with its own certificate obtains a *different* τ and cannot reuse the victim's ML-DSA signature; a cross-chain replay sees a different context. This is the QUIC analogue of luxd's existing chain-ID-bound PQHandshake (§3.3); we reuse that design rather than invent one.

6.3 The hybrid NodeID

We preserve the classical value and pair it with the PQ value.

Definition 1 (Hybrid validator identity). *A validator's identity is the pair $(\text{NodeID}_{\text{cls}}, pk_{\text{pq}})$ where*

$$\text{NodeID}_{\text{cls}} = \text{RIPEMD160}(\text{SHA256}(\text{cert.Raw})), \quad pk_{\text{pq}} = \text{ML-DSA-65 public key},$$

¹Terminology: colloquially "X-Wing." Precisely, the TLS group X25519MLKEM768 uses the TLS-hybrid concatenation KDF, which is closely related to but not identical to the single-KEM X-Wing construction of ePrint 2024/039 [1]. Both are secure hybrid KEMs; we cite the analysis for the combiner family [4].

cross-bound by two signatures published at registration: the staking key signs pk_{pq} and pk_{pq} signs $NodeID_{cls}$, each under a domain-separated context. The active $NodeID$ under a PQ profile is $NodeID_{pq} = NodeIDSchemeMLDSA65.DeriveMLDSA(chainID, pk_{pq})$, the scheme *luxd* already ships (§3.3).

Claim 1 (Identity continuity, G3). *Under **ProfilePermissive**, the validator-set key remains $NodeID_{cls}$ (20 bytes), so no stake-map migration is forced; the ML-DSA leg is additive authentication. Under **ProfileStrictPQ**, the key is $NodeID_{pq}$, and the cross-binding lets verifiers that still track $NodeID_{cls}$ follow the transition without a flag day.*

Claim 2 (Hybrid unforgeability, G2). *To present as validator V on a QUIC-PQ connection an adversary must produce a valid channel-bound ML-DSA-65 signature under V 's pk_{pq} and complete a TLS handshake with V 's staking certificate. Under EUF-CMA of ML-DSA-65 (FIPS-204 [9]) the first is infeasible without sk_{pq} ; a quantum forger who breaks only the classical staking key still cannot forge the ML-DSA leg. Conversely a lattice adversary who forges ML-DSA but not the classical credential fails the TLS leg under **ProfilePermissive**. Impersonation requires breaking both — the X-Wing pattern for identity.*

6.4 Handshake flow (design)

1. QUIC/TLS 1.3 handshake; negotiate X25519MLKEM768, ALPN "zap/1". [live]
2. Both sides derive τ from the TLS exporter (§6.2).
3. Client opens the identity stream (bidi, class 'C', id 0), sends $\langle pk_{pq}, sig_I, NodeID_{cls}, \text{cross-bind sigs} \rangle$.
4. Server verifies sig_I over $(H_2, \tau, \text{role} = I)$, checks $pk_{pq} \rightarrow NodeID$ under the active scheme, checks validator-set membership, then replies with its own $\langle pk_{pq}, sig_R, \dots \rangle$.
5. Client verifies symmetrically. On success both sides hold a mutually-authenticated, PQ-confidential, channel-bound connection whose $NodeID$ is trustworthy. Consensus streams (Class C, 1-RTT only) may now open.

Reuse map: step 1 is *luxfi/zap/quic* verbatim; the AUTH signature/verify in steps 3–5 is *zap/handshake.Identity.Sign/VerifyAuth* with τ added to the sign-input; the $NodeID$ derivation is *luxd*'s *NodeIDSchemeMLDSA65*. No new cryptographic code, only the binding of τ and the stream plumbing.

7 Consensus-P2P over QUIC-ZAP

7.1 The message set is unchanged

The Lux/Simplex message set (*proto/p2p*) is carried byte-for-byte. We change *where the bytes travel*, not the bytes. Table 2 gives the stream map.

Category	Messages	QUIC stream
Handshake/net	Handshake, GetPeerList, PeerList, Ping, Pong	uni (gossip); identity on bidi id 0
State-sync	GetStateSummaryFrontier, StateSummaryFrontier, GetAcceptedStateSummary, AcceptedStateSummary	bidi req/resp
Bootstrap	GetAcceptedFrontier, AcceptedFrontier, GetAccepted, Accepted, GetAncestors, Ancestors	bidi req/resp (own stream: large, HOL-isolated)
Consensus	Get, Put, PushQuery, PullQuery, Chits	bidi req/resp, 1-RTT
App	AppRequest, AppResponse, AppGossip, AppError	bidi (req/resp) / uni (gossip)
Simplex	BlockProposal, Vote, EmptyVote, Notarization	uni (gossip), 1-RTT

Table 2: Lux/Simplex messages to QUIC stream classes. Semantics are preserved: a request expects one response; gossip expects none. Large bootstrap transfers get their own streams so they cannot HOL-block votes (§5.3).

7.2 Drop-in compatibility via the `net.Conn` seam

luxd’s peer layer is already transport-abstract. Inbound goes through

```
Upgrader.Upgrade(net.Conn) → (NodeID, net.Conn, *Certificate, err),
```

and outbound through `EndpointDialer` (which already carries TCP, DNS, and an RNS mesh transport). Two integration levels:

- L0 — Conservative drop-in.** A QUIC Upgrader completes the handshake of §6.4 and returns a `net.Conn` that *is* one bidirectional QUIC stream. The existing message read/write loop runs over it unchanged; message framing is byte-identical to TCP. This yields PQ security and the single-port collapse with *zero* change to the consensus message loop — the minimal, safest first step, and the one the A/B test (§10) should gate on.
- L1 — Full multiplex.** The peer layer is taught to open a stream per message class (Table 2), unlocking the no-HOL win (§5.3) and per-Call RPC parallelism. This is a peer-layer change *above* the transport and *below* BFT; it does not alter message semantics. It is opt-in and can ship after L0 is proven.

Crucially the Upgrader contract already returns the `(NodeID, net.Conn, cert)` triple the network expects, so the consensus layer (`network.NewNetwork`) is oblivious to which transport produced the conn. This is the decompletion that makes the change safe.

7.3 Reusing the node2 vote-mesh

The Simplex Vote/Notarization path is precisely what `node2` already runs: a `SignedVote` (`block_id || voter || sig`) gossiped as a ZAP frame, verified and de-duplicated by the gate, finalised by a $> 2/3$ -stake quorum certificate. Over QUIC-ZAP the same `encode_vote` / `decode_vote` codec (with its strict no-trailing-bytes non-malleability check) is written to a Class C unidirectional stream instead of a raw socket, and `FrameReader`’s hand-rolled non-blocking reassembly is subsumed by QUIC’s per-stream delivery. The proven wire semantics (§3.4) are unchanged; only the byte carrier changes. This is the C++ vote-mesh and the Go peer layer meeting at the same ZAP frame.

8 Fallback negotiation and mixed fleets

8.1 Baseline and capability

The proven classical TCP staking listener on `9631/tcp` is the permanent baseline. A validator additionally binds `9631/udp` for QUIC-PQ. QUIC-PQ is a *capability* discovered three ways (any one suffices):

1. **Handshake capability bit.** The Handshake/Version message gains an optional `transports` field advertising `quic-pq/1`. Absent the field, the peer is TCP-only. (A new *optional* field is backwards-compatible with the proto.)
2. **DNS advertisement.** A SRV/TXT record (§9) publishes the UDP port and `quic-pq/1`.
3. **mDNS** on local segments (`luxfi/zap` already ships mDNS discovery), for datacenter/local-fiber clusters.

8.2 Upgrade rule

Peer A	Peer B	Result
QUIC-PQ	QUIC-PQ	QUIC-PQ (both advertise; dial UDP; TCP idle)
QUIC-PQ	TCP-only	TCP 9631 (proven path)
TCP-only	QUIC-PQ	TCP 9631
TCP-only	TCP-only	TCP 9631

The rule is monotone: presence of the capability on *both* sides is necessary and sufficient to use QUIC-PQ; otherwise the fleet transparently uses the classical path. A validator runs both listeners concurrently, so the fleet is a free mix of upgraded and legacy nodes with no partition (G6). Anti-downgrade: once both peers have advertised `quic-pq/1` (a value carried *inside* the authenticated Handshake on whichever transport bootstrapped them), a silent downgrade to TCP by a network adversary is detectable — the peers expected QUIC and can log/refuse per policy. Because both transports deliver the *same* authenticated message stream to the same BFT logic, correctness never depends on which was chosen; the anti-downgrade check is a monitoring and policy affordance, not a safety dependency.

8.3 Migration discipline

TCP 9631 is retired for a network *only* after the A/B gate (§10) shows byte-identical finality on that network for a sustained window. Order: devnet → testnet → mainnet, each gated independently. This mirrors the estate’s standing rollout discipline (canary, then atomic ≥ quorum).

9 DNS scheme and validator onboarding

9.1 One name per validator per network

Network	Zone	Validator name
mainnet	<code>lux.network</code>	<code><NN>.lux.network</code>
testnet	<code>lux-test.network</code>	<code><NN>.lux-test.network</code>
devnet	<code>lux-dev.network</code>	<code><NN>.lux-dev.network</code>

NN is a zero-padded index (01, 02, ...); local fiber DEX validators occupy 06 and up. Each name resolves to:

- **A/AAAA** — the validator’s public IP;
- **SRV _zap-quic._udp.<NN>.<zone>** — the QUIC-PQ UDP port and priority/weight;
- **TXT** — capability string `quic-pq/1`, and the *fingerprint* of the validator’s pk_{pq} (SHA3-256), so a resolver can pin the expected ML-DSA identity before dialing (DNS is advisory; the handshake is authoritative).

DNS is a *discovery* convenience only. Peer identity is never DNS — it is the channel-bound ML-DSA NodeID of §6. A poisoned DNS answer can at worst deny service (wrong IP) or trigger a handshake that fails identity verification; it cannot cause acceptance of a wrong validator.

9.2 Local-fiber / NAT validator join

A validator behind a home/office fiber uplink (the 06+ local DEX validators) joins as follows:

1. **Key + identity.** Generate the staking cert and the ML-DSA-65 keypair; compute $(NodeID_{cls}, pk_{pq})$ and the cross-binding signatures (Def. 1).
2. **Register on P-chain.** `AddPermissionlessValidator` with the hybrid identity (the P-chain tx carries the staking key; pk_{pq} is bound per §3.3). This is unchanged from today except that the PQ key is included.
3. **Publish endpoint.** Set A/SRV/TXT for `<NN>.lux-dev.network` (or the operator’s own name; the NN scheme is a convention, and TXT capability discovery works under any name).
4. **Reachability.** QUIC is UDP, so a single inbound UDP port forward on the CPE suffices; QUIC connection migration (§5.4) tolerates the periodic public-IP change of a residential uplink without dropping consensus connections. Where no inbound port is possible, the validator dials out to already-reachable peers and QUIC keeps the bidirectional connection open (consensus is symmetric over an established conn); NAT hole-punching via a

rendezvous peer is a future option but not required for the common single-port-forward case.

5. **Peer over QUIC-PQ.** Discover peers via DNS/mDNS, dial their SRV UDP endpoints, complete the §6.4 handshake, and begin exchanging the (unchanged) consensus messages.

10 Devnet A/B acceptance: byte-identical finality

The retirement gate for the fallback is an empirical equality, not an argument. Run the *same* validator set finalising the *same* chain twice:

Run A all consensus-P2P over TCP 9631 (baseline).

Run B all consensus-P2P over QUIC-PQ (§7, at integration level L0 first, then L1).

Definition 2 (Acceptance). *Runs A and B pass iff, for every height h in the test window, they agree on: the accepted block id and full block bytes at h ; the accepted frontier; and the finality certificate bytes (the BLS/Simplex quorum certificate, including the vote set and aggregate signature). Any divergence is a fail.*

Because the transport is strictly below the message layer and the message bytes are identical between A and B, finality *must* be identical; the test *proves* the transport introduced no semantic change (any difference indicts the transport). The C++ analogue is already green: `node2`'s `cluster_5` finalises with a verifying $> 2/3$ -stake certificate over the ZAP mesh (§3.4); the Go devnet A/B lifts that guarantee to the full luxd message set.

Instrumentation. Compare via the finality ledger's per-height certificate bytes and `LastAccepted` at matched heights; diff block bytes by hash. Run each arm ≥ 3 times to rule out nondeterministic scheduling; require bitwise equality of the certificate set (order-normalised by validator index, as the certificate encoding already prescribes).

11 BFT preservation argument

Assumption 1 (Message-layer BFT, unchanged). *Under partial synchrony [3] with $< n/3$ Byzantine stake and an authenticated network that eventually delivers messages between honest parties within Δ after GST, the Lux/Simplex protocol is safe (no two honest nodes finalise conflicting blocks at a height) and live (honest transactions eventually finalise). This is proven at the message layer (LP-020 [5]) and is not re-proven here.*

Theorem 1 (Transport transparency). *Replacing the TCP staking transport with QUIC-PQ (§7, integration L0 or L1) preserves the safety and liveness of Assumption 1.*

Proof sketch. The BFT protocol consumes a stream of *authenticated messages* and requires two properties of the transport: (P1) *authentication* — a message attributed to honest V was sent by V ; (P2) *eventual delivery* — after GST, a message between honest parties arrives within Δ . We show QUIC-PQ provides both at least as strongly as TCP-TLS.

Safety. Safety depends only on (P1) and on the message contents, not on ordering across distinct messages (each `Vote/Chits` is independently attributable and idempotently counted; the gate de-duplicates by voter key, exactly as `node2` does). QUIC-PQ strengthens (P1): every message rides a connection whose `NodeID` is bound by a channel-bound ML-DSA-65 signature (§6.2), which is $\geq$ the TLS-cert binding of the classical path and additionally PQ-secure. The consensus message bytes are identical (§7), so the set of finalisable decisions is unchanged. The one transport-level way to *violate* (P1) would be actionable replay; we forbid 0-RTT on Class C (§5.4), so no consensus message is delivered except as authenticated 1-RTT data. Hence no new equivocation or double-count is possible, and safety is preserved.

Liveness. QUIC provides per-stream reliable, ordered delivery — the same guarantee TCP gives per connection — so after GST honest messages are delivered within a Δ no larger than TCP’s (QUIC’s loss recovery is comparable to TCP’s, and its handshake is $\leq$ one extra flight, amortised by 0-RTT for idempotent RPC). Moreover per-stream independence *removes* cross-message HOL blocking (§5.3): a stalled bootstrap transfer no longer delays a consensus vote, which can only *decrease* effective message latency under load. Connection migration (§5.4) preserves liveness across address changes that would drop a TCP connection and force re-dial. Thus (P2) holds with a Δ no worse — typically better — than the baseline, and liveness is preserved.

Since both required properties are met, Assumption 1’s conclusions carry over unchanged.

□

□

Claim 3 (The $f < n/3$ bound is untouched). *The fault threshold is a property of the decision rule and the validator set, both unchanged. The transport neither adds nor removes validators, neither weakens nor strengthens the quorum arithmetic; it only carries the same votes among the same set. The A/B test (§10) is the empirical confirmation of Theorem 1.*

12 Minimal proof-of-concept

Two cheap, self-contained artifacts de-risk the design without touching consensus:

1. **QUIC-PQ echo + CurveID assertion.** A client/server that opens a `luxfi/zap/quic` connection, asserts `ConnectionState().CurveID == 0x11EC`, opens the three stream classes (R/C/Z), writes a class-prefixed frame on each, and echoes it back — proving the demux (§5.2) and that PQ key agreement is really negotiated. (The CurveID assertion already exists in `quic`’s test suite; the PoC adds the three-class prefix and round-trips one frame per class.)
2. **Channel-bound AUTH over a QUIC stream.** On stream id 0, run `zap/handshake.Identity.Sign/V` with the RFC-5705 exporter τ folded into the sign-input (§6.2); assert that a signature produced against one connection’s τ fails verification against another connection — the channel-binding property (G4), demonstrated as a negative test.

Both live in `luxfi/zap` (transport + handshake) and require no change to `luxd`’s consensus or proposer VM, matching the scope guardrail. They are the executable core of §5–§6; the `luxd Upgrader/EndpointDialer` wiring (§7.2) and the A/B harness (§10) are the follow-on integration.

13 Related Lux work

This paper is the transport capstone of the ZAP stack. LP-200 [6] is the ten-layer umbrella; LP-201 [7] is the universal wire format whose frames ride these streams; LP-9702 [8] is the Z-Wing hybrid PQ secure channel we reuse as the identity layer; LP-020 [5] is the consensus whose messages this transport carries unchanged. The `node2/consensus2` vote-mesh is the C++ realisation of the Class C gossip path. Against upstream Avalanche, the sole delta is the transport and the hybrid identity binding; the message set and BFT logic are preserved verbatim (the “extend, don’t break” guardrail).

14 Conclusion

One UDP port per validator, QUIC over ZAP, end-to-end post-quantum: RPC, consensus-P2P, and ZAP-native services multiplexed as independent QUIC streams over an `X25519MLKEM768` channel with a channel-bound ML-DSA-65 hybrid `NodeID`. The design is composition over invention — every primitive is already proven and live; the new work is a channel binding, a

three-class demux, and the luxd transport-seam wiring. The classical TCP path remains the negotiated fallback until a devnet A/B test proves byte-identical finality, and a message-layer argument shows the transport swap cannot alter $f < n/3$ safety or liveness. The result collapses three endpoints and three security postures into one, removes transport head-of-line blocking from the consensus path, and makes every validator connection quantum-safe by default — without touching a line of the BFT decision logic.

References

- [1] M. Barbosa, D. Connolly, J. Duarte, A. Kaiser, P. Schwabe, K. Varner, and B. Westerbaan. X-Wing: The hybrid KEM you’ve been looking for. IACR ePrint 2024/039, 2024.
- [2] N. Bindel, J. Brendel, M. Fischlin, B. Goncalves, and D. Stebila. Hybrid key encapsulation mechanisms and authenticated key exchange. *Post-Quantum Cryptography (PQCrypto 2019)*, LNCS 11505, pages 206–226, 2019.
- [3] C. Dwork, N. Lynch, and L. Stockmeyer. Consensus in the presence of partial synchrony. *Journal of the ACM*, 35(2):288–323, 1988.
- [4] F. Giacon, F. Heuer, and B. Poettering. KEM combiners. In *Public-Key Cryptography (PKC 2018)*, LNCS 10769, pages 190–218. Springer, 2018.
- [5] Lux Industries, Inc. LP-020: Quasar consensus. Lux Proposal LP-020, 2026.
- [6] Lux Industries, Inc. LP-200: The lux ZAP stack — a decomplexed ten-layer blockchain architecture. Lux Proposal LP-200, 2026.
- [7] Lux Industries, Inc. LP-201: The ZAP universal wire format. Lux Proposal LP-201, 2026.
- [8] Lux Industries, Inc. LP-9702: Z-Wing — a hybrid post-quantum secure channel. Lux Proposal LP-9702, 2026.
- [9] National Institute of Standards and Technology. Module-lattice-based digital signature standard. FIPS 204, 2024.
- [10] E. Rescorla. Keying material exporters for transport layer security (TLS). RFC 5705, IETF, 2010.
- [11] M. Thomson and S. Turner. Using TLS to secure QUIC. RFC 9001, IETF, 2021.